

Software Design Report

Project: 1757 - A Digital Social Deduction Game

Submitted To:
Farzana Tabassum
Lecturer, Dept. of CSE
Islamic University of Technology

Supervisor:
Samia Nawsheen
Junior Lecturer, Dept. of CSE
Islamic University of Technology

Team Members (Group ID: 1757)

Name	Student ID
Md. Saif-Al Sarker	230042106
M Safwan Hasan Khan	230042117
Wasi Omar Syed	230042143

July 26, 2026

1 Introduction

This report details the core software design of **1757**, a real-time multiplayer social deduction game set against the historical backdrop of the Battle of Plassey. Because maintaining complex web applications requires strict structural separation, the system follows a classic Object-Oriented design integrated with modern web frameworks. We present the system using standard design models to highlight its robustness, scalability, and ease of maintainability.

2 Software Architecture

2.1 3-Tier Architecture Diagram

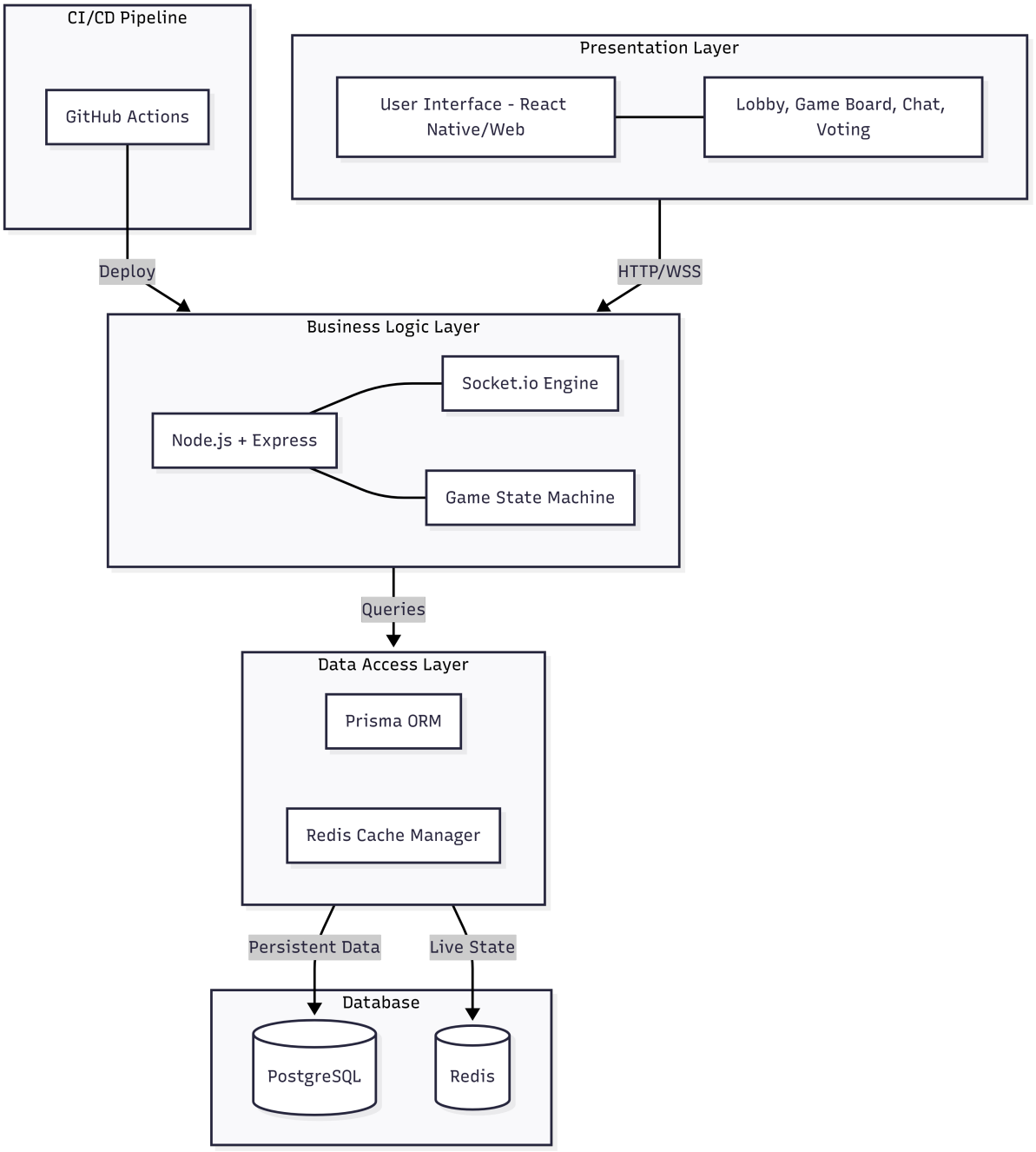


Figure 1: 3-Tier Architecture of 1757

2.1.1 Explanation of Architecture

The system leverages a **3-Tier Architecture**, keeping user interaction completely isolated from backend processing and data storage. This Separation of Concerns ensures that changes to the user interface do not compromise the integrity of game rules.

Presentation Layer: This layer handles client interaction across Web and Mobile via React Native. It provides the visual interfaces—such as the lobby, game board, real-

time chat, and voting overlays. It interacts with the backend over HTTP and continuous WebSocket (Socket.io) connections to ensure immediate feedback.

Business Logic Layer: At the core, an Express server built on Node.js implements the game's logic. Within this layer, a specialized Finite State Machine (FSM) governs game flow, guaranteeing that phases strictly transition without overlapping. It listens for Socket.io events (e.g., placing votes) and resolves them immediately. A GitHub Actions pipeline automatically manages continuous deployment to this layer.

Data Access Layer & Database: PostgreSQL (via Prisma ORM) persistently stores player accounts, profiles, and historical game results. Because social deduction games require sub-millisecond responses for hundreds of concurrent connections, Redis is employed here. Redis caches highly volatile data, such as active lobbies, real-time vote tallies, and temporary character state variables, acting as the immediate memory for the game engine.

3 Structural Modeling

3.1 Class Diagram

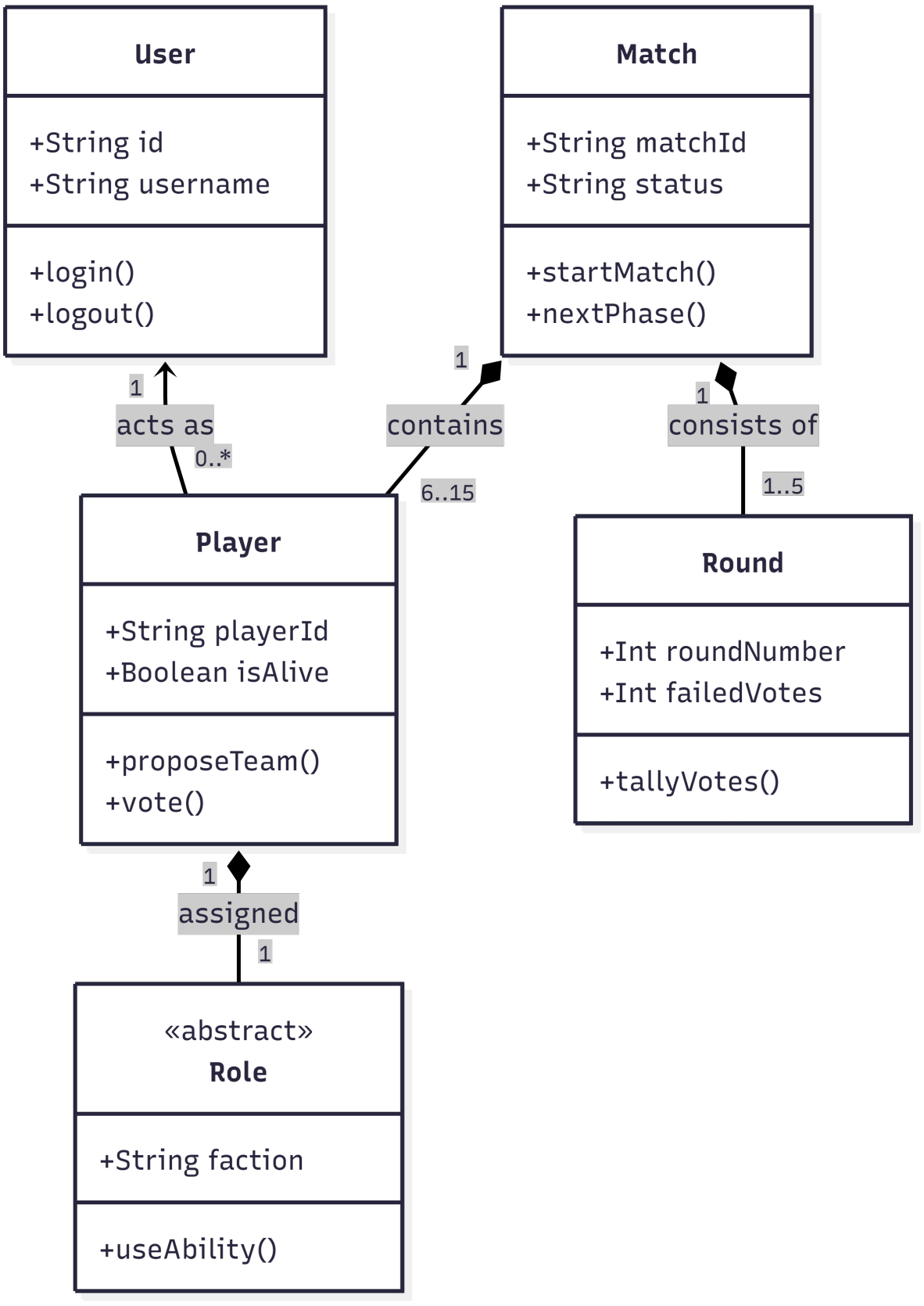


Figure 2: Class Diagram mapping Object-Oriented entities

3.1.1 Explanation of Class Design

The object-oriented design transforms the game's abstract rules into concrete data structures. The `User` class represents registered individuals holding authentication credentials. When entering a game session, a `User` acts as a `Player`.

The `Match` class serves as the root composite object aggregating multiple `Player` instances (between 6 and 15) and several `Round` instances. It dictates the current phase of the match and encapsulates methods for advancing states. The abstract `Role` class is assigned to a Player, defining baseline traits such as their allegiance (Nawab or EIC), and is extended by specific character classes handling custom abilities. The `Round` class specifically deals with the micro-mechanics of tallying votes and finalizing missions, isolating those details from the overarching match logic.

4 Behavioral Modeling

4.1 Use Case Diagram

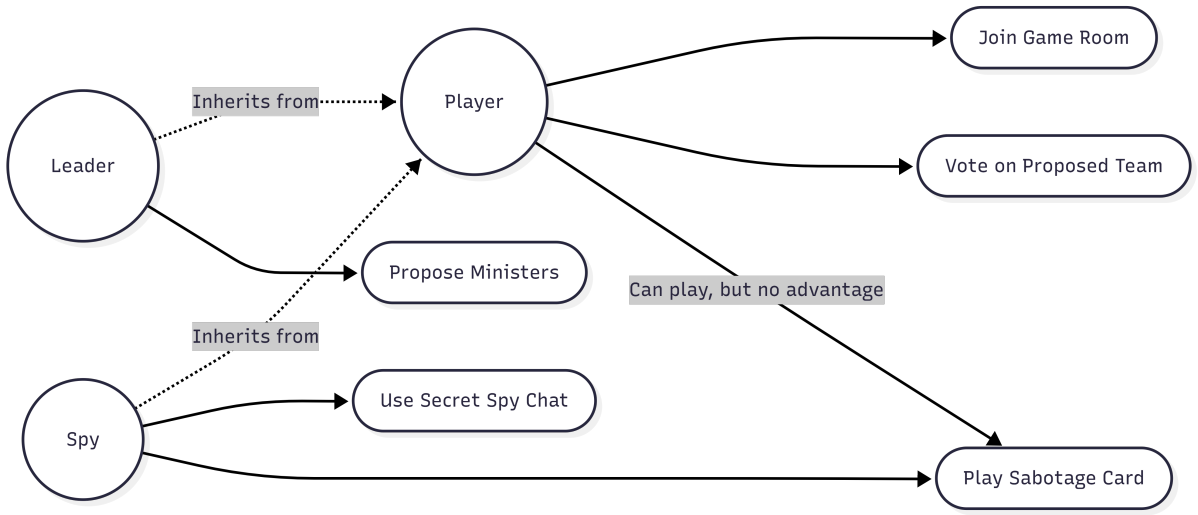


Figure 3: Player interaction Use Case Diagram

4.1.1 Explanation of Use Cases

The Use Case diagram identifies the various interactions distinct actors can perform within the system. The generic `Player` actor can join matchmaking rooms, propose teams (if they become the Leader), and cast their ballot on those proposed teams.

Sub-actors like `Spy` inherit all basic player abilities but also have access to exclusive, encrypted interactions, such as using the dark room chat to conspire privately or playing sabotage cards during missions. Although all players mechanically have the ability to play a "Sabotage" card on a mission, doing so provides no strategic advantage to Nawab Loyalists.

4.2 State Machine Diagram

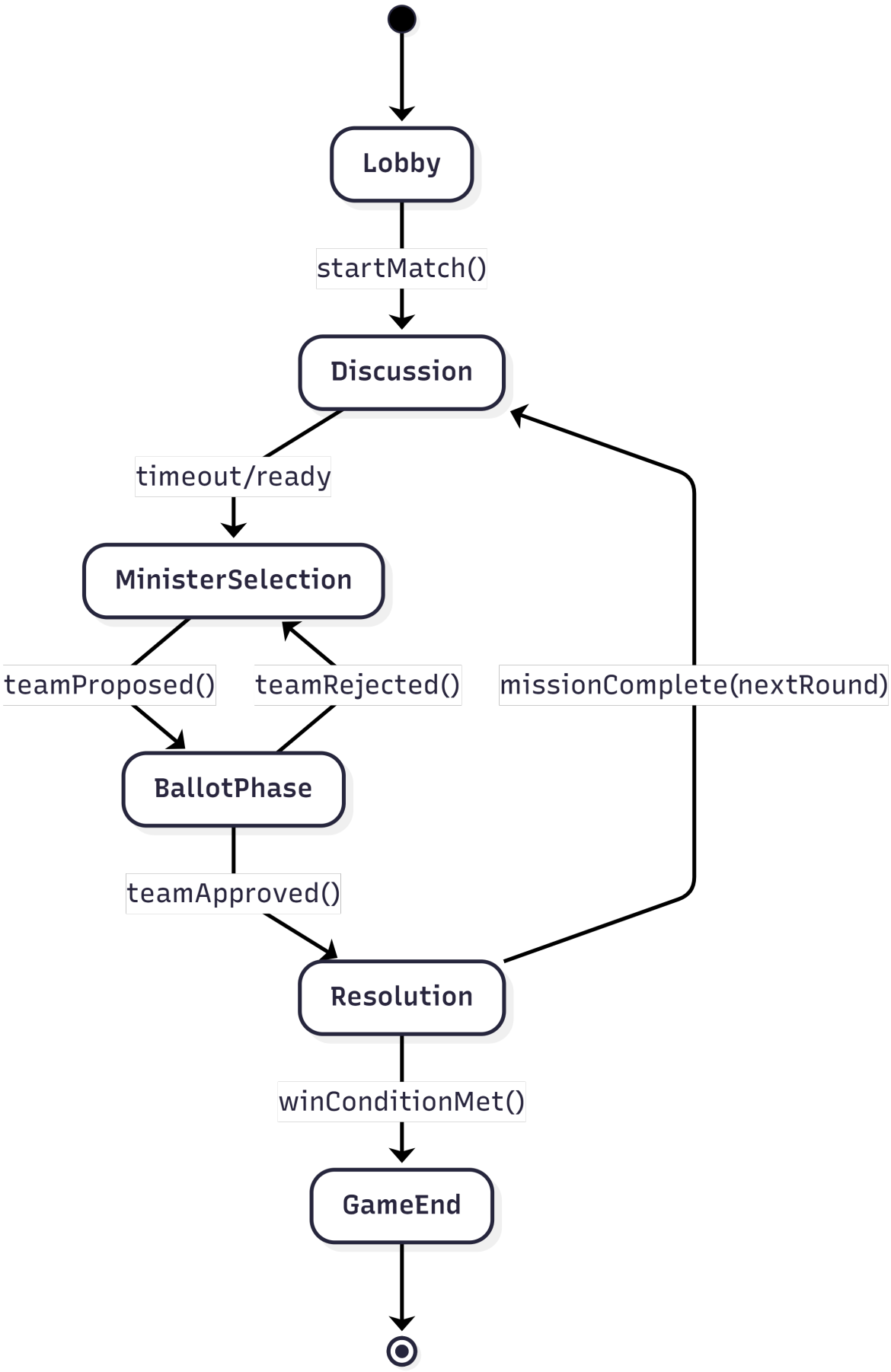


Figure 4: Finite State Machine representing phase transitions

4.2.1 Explanation of State Machine

Because **1757** relies on a rigid progression of phases, a Finite State Machine (FSM) dictates the boundaries of a match. The state begins in the **Lobby**, waiting for participants. Once initiated, it transitions into **Discussion**, where players debate.

Upon a timeout or mutual readiness, the system strictly moves to **Minister Selection**, where only the designated Leader can act. Proposing a team triggers the **Ballot Phase**, forcing all players to vote. Rejection immediately loops the state back to Selection (handing leadership to the next player). Approval advances the state to **Resolution**, where the mission is played out, ultimately looping back to Discussion or landing on **GameEnd** if a faction achieves their win conditions.

4.3 Sequence Diagram

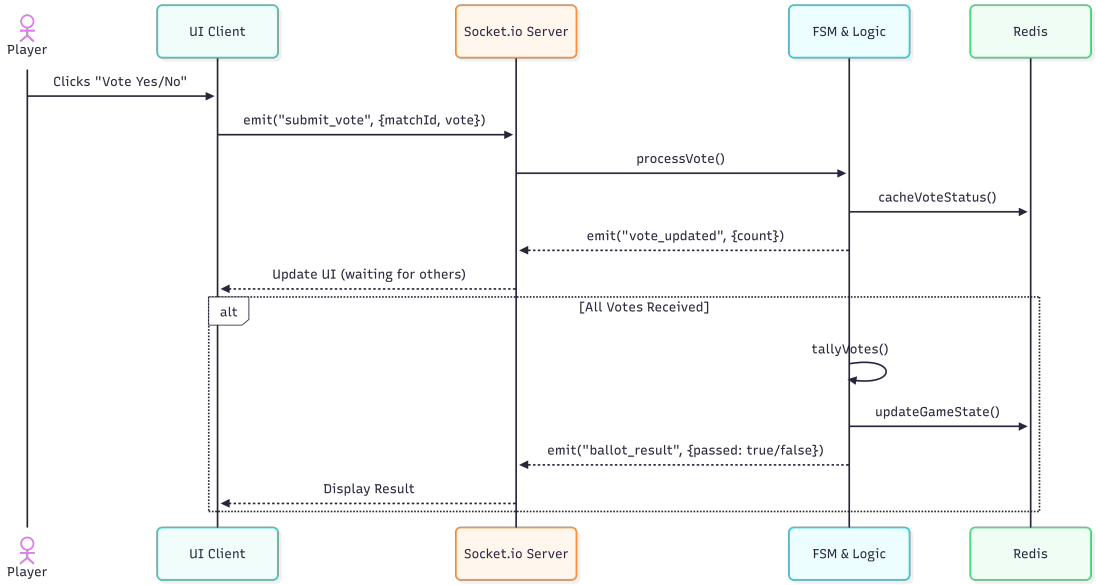


Figure 5: Sequence of events during real-time Voting

4.3.1 Explanation of Sequence Flow

The sequence diagram illustrates the complex backend synchronization required during the Ballot Phase. When a player casts a vote, their client immediately fires an event to the Node.js WebSocket Server.

Instead of burdening the PostgreSQL database with rapid I/O operations, the Game Logic passes this incoming vote to the Redis cache. The server instantly emits a payload updating the UI of all connected clients to indicate a vote was placed (without revealing what the vote was). Once Redis confirms all expected ballots have been cast, the FSM resolves the total count, updates the overarching game state, and signals the clients to transition to the result screen, ensuring real-time harmony across all devices.

4.4 Activity Diagram

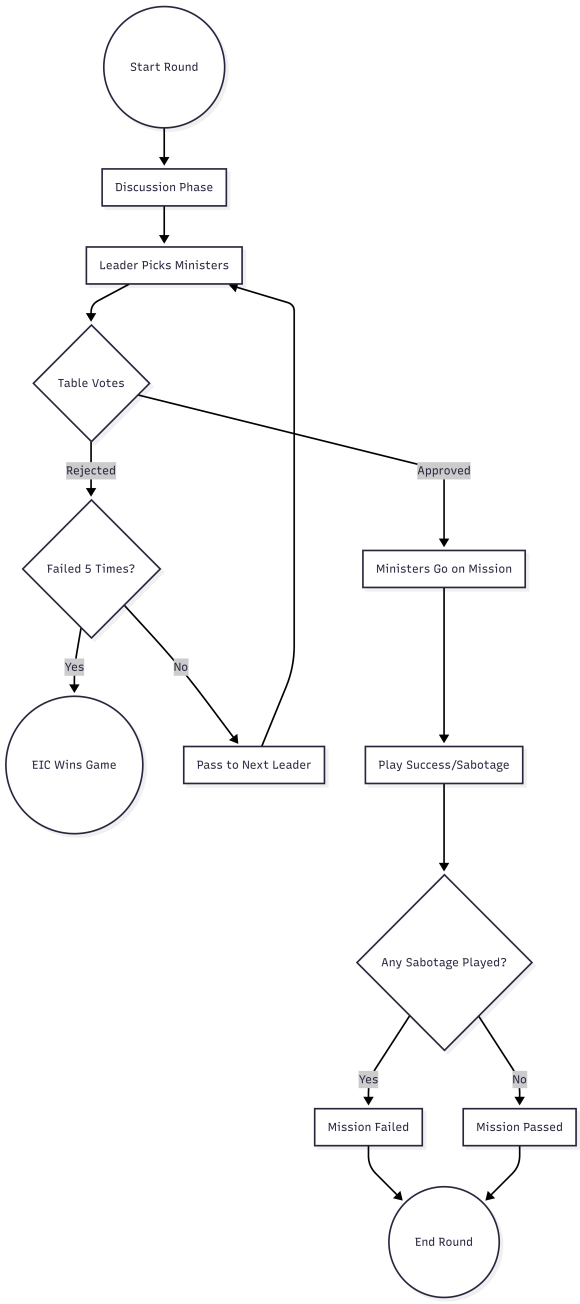


Figure 6: Activity flow for a single round

4.4.1 Explanation of Activity Flow

The Activity Diagram charts the procedural flow of a single round from start to finish. The round begins with an open Discussion, leading to the current Leader selecting ministers.

A critical decision node appears during the table vote. If the team is rejected, the flow checks if 5 rejections have occurred sequentially. If true, the game abruptly ends in an EIC victory due to internal chaos. Otherwise, leadership passes, and the selection repeats. If the team is approved, the chosen ministers embark on the mission and submit

cards. The flow then checks for the presence of any sabotage cards; a single sabotage fails the mission for the Loyalists. This visualizes the core algorithmic loop the FSM follows.

5 Database Design

5.1 Entity-Relationship (ER) Diagram

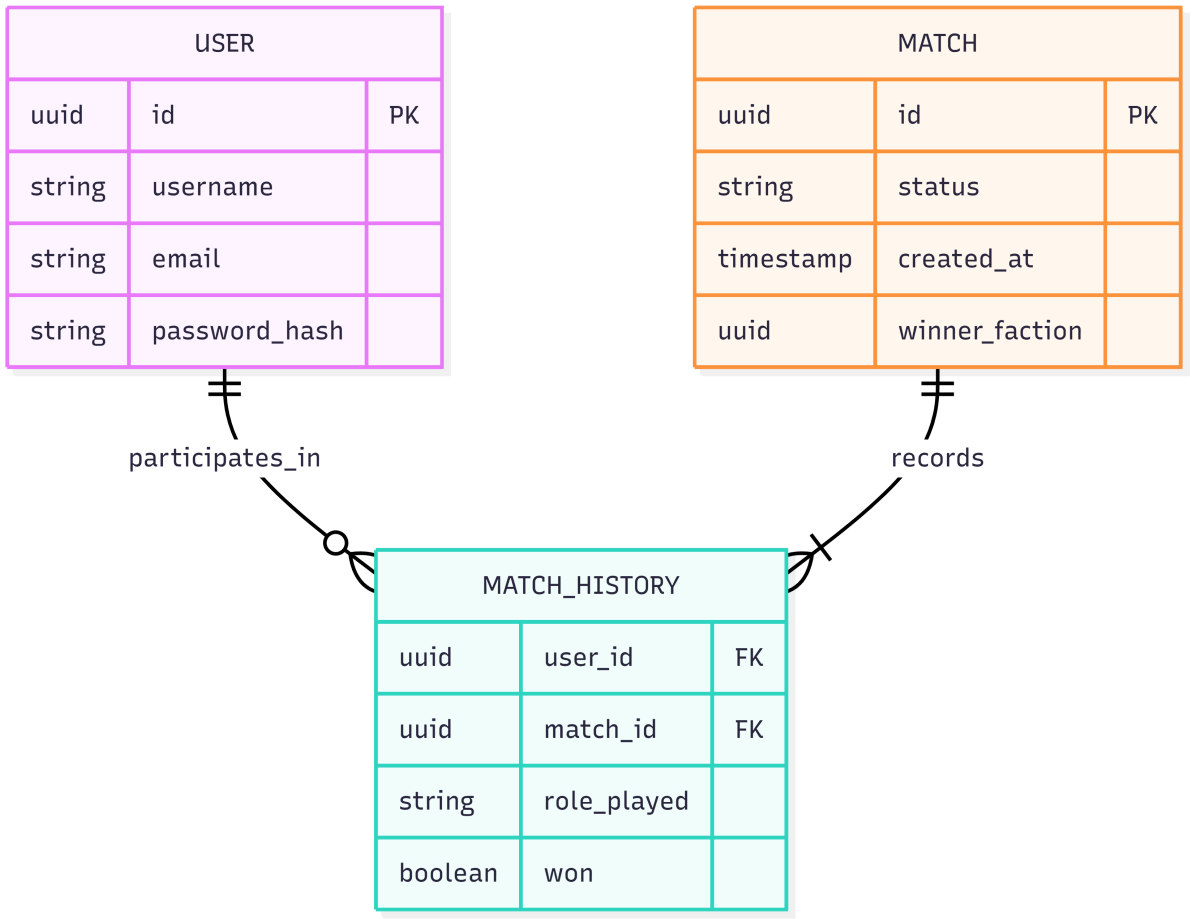


Figure 7: ER Diagram for persistent PostgreSQL storage

5.1.1 Explanation of ER Schema

To optimize performance and reduce database bloat, the persistent PostgreSQL database strictly maintains only long-term, non-volatile data. The ER diagram maps out this simplified structure.

The `USER` entity stores authentication credentials and unique identifiers. The `MATCH` entity simply logs the final outcome of a session, storing the date it occurred and the faction that ultimately won. The associative entity, `MATCH_HISTORY`, bridges the many-to-many relationship. It records exactly which users participated in which matches, noting their specific role and whether they were on the winning side. This allows the platform to generate leaderboards, win rates, and user statistics without storing the heavy, granular details of every vote or chat message.